

Ролевые метки или дополнительные вызовы?

Контролируемое исследование качества и стоимости генерации кода большими языковыми моделями

ANONYMOUS AUTHOR(S)

При генерации кода большие языковые модели могут получать одну прямую инструкцию или последовательность этапов «планирование — реализация — проверка». Названия ролей часто сочетаются с дополнительными вызовами, что затрудняет оценку каждого решения по отдельности. Мы сравниваем нейтральные и ролевые названия этапов при выполнении одинаковых операций в одном или трех вызовах; пятым условием служит прямой решатель. Эксперимент с GPT-5.6 Luna включает 15 000 назначений на 1 000 задачах BigCodeBench — набора задач на генерацию функций Python с автоматическими тестами. Завершена 14 961 генерация; для 985 задач пригодность тестового окружения подтверждена контрольными программами. Качество измеряется долей программ, прошедших исходные тесты, а ресурсы — токенами и их оценкой по тарифам программного интерфейса поставщика (API). Ролевые метки изменяют парную долю успеха на $-0,75$ процентного пункта при одном вызове и на $+1,35$ пункта при трех; с учетом множественных статистических проверок надежного преимущества по качеству не установлено. Три вызова снижают эту долю на 4,17 и 2,13 пункта соответственно для нейтральных и ролевых инструкций, увеличивая стоимостную оценку в 2,67 и 2,78 раза. Направление различий структуры вызовов сохраняется на 800 новых задачах и совпадает с более ранней серией на 200 задачах с GPT-5.4-mini. В отдельном сравнении 2 000 блоков Self-Collaboration уступает одношаговым процессам по качеству на наблюдаемых парах и имеет примерно втрое большую стоимостную оценку. Для всех назначений дополнительно рассчитаны границы с учетом отсутствующих исходов: они подтверждают отрицательное направление различий структуры вызовов в основной серии, а для Self-Collaboration допускают оба порядка методов. Результаты дают основание использовать прямой и одношаговый поэтапный процессы как экономичные базовые варианты в исследованном режиме полного контекста; добавление ролей или вызовов требует отдельного подтверждения пользы.

CCS Concepts: • **Software and its engineering** → **Automatic programming**; *Software testing and debugging*.

Additional Key Words and Phrases: генерация кода большими языковыми моделями, оркестрация промптов, ролевые промпты, факторный эксперимент, исполнимая проверка, учет ресурсов

ACM Reference Format:

Anonymous Author(s). 2027. Ролевые метки или дополнительные вызовы? Контролируемое исследование качества и стоимости генерации кода большими языковыми моделями. In *ACM International Conference on the Foundations of Software Engineering (FSE 2027)*, 12–16 July 2027, Shenzhen, China. ACM, New York, NY, USA, 25 pages.

1 ВВЕДЕНИЕ

1.1 Предметная область и актуальность

Большие языковые модели (БЯМ; английское название — large language models, LLM) создают текст, в том числе программный код, по заданной инструкции. В задачах программной инженерии на вход модели передают требование и контекст: например, заготовку функции, описание доступных библиотек и ограничения задачи. Такой вход вместе с инструкцией называют *промптом*. Один вызов модели — это отправка входа и получение одного

FSE 2027, 12–16 July 2027, Shenzhen, China
2027.

ответа. Модель обрабатывает текст в токенах — фрагментах слов, знаках пунктуации и элементах кода; число входных и выходных токенов используют для учета ресурсов и расчета стоимости обращения к программному интерфейсу поставщика (API).

Получение программы можно организовать по-разному: сразу попросить решение или предложить последовательно составить план, написать код и проверить его. *Ролевая метка* — название функции, приписанной этапу инструкции, например «планировщик», «разработчик» или «рецензент». Для сравнения тот же этап можно назвать нейтрально: «этап 1», «этап 2» или «этап 3». Сама метка не создает отдельного агента: три роли могут быть перечислены в одном промпте либо распределены между отдельными вызовами. Под *структурой вызовов*, или топологией, далее понимается число вызовов и порядок передачи ответов между ними.

Системы Self-Collaboration, ChatDev, MetaGPT, AgentCoder и PairCoder объединяют роли с обменом сообщениями, генерацией тестов или исправлением решений [5, 7, 9, 19, 26]. Такие процессы могут быть полезны, когда следующий этап получает новые сведения или результаты выполнения программы. Одновременно дополнительный вызов может повторно передавать весь контекст и промежуточные ответы. Поэтому выбор организации процесса влияет не только на качество, но и на **стоимость генерации кода**.

1.2 Проблема исследования

При сравнении целых систем обычно одновременно меняются названия ролей, содержание инструкций, число вызовов, доступ к инструментам и объем вычислений. Поэтому различие результатов не показывает, какой именно компонент принес пользу или дополнительные затраты. Практическая проблема состоит в отсутствии изолированной оценки двух простых проектных решений: нужно ли называть этапы ролями и нужно ли выполнять одни и те же операции в отдельных вызовах. Без такой оценки разработчик может усложнить процесс и увеличить расход ресурсов, не получив более качественную программу.

В этой работе *успешным решением* считается итоговая программа, прошедшая все исходные автоматические тесты выбранной задачи в пригодном для проверки окружении. *Доля успешных решений*, далее также *успешность*, — доля таких исходов среди оцениваемых генераций. Пригодность окружения проверяют заранее: эталонная программа должна пройти тесты, а заведомо неверная — не пройти. Успех по тестам является измеримой характеристикой качества для данного набора задач; он не означает доказанную корректность программы на любых возможных входах.

1.3 Цель, исследовательские вопросы и задачи

Цель работы — определить, как ролевые метки и распределение неизменных операций между одним и тремя вызовами языковой модели связаны с долей успешных решений и расходом ресурсов при генерации кода с полным контекстом задачи.

Из этой цели следуют три исследовательских вопроса (RQ, research question):

- RQ1.** Изменяют ли ролевые метки долю успешных решений, если содержание операций и структура вызовов остаются неизменными?
- RQ2.** Как выполнение этих операций в трех вызовах вместо одного изменяет долю успешных решений, число токенов и стоимостную оценку?
- RQ3.** Как поэтапные процессы соотносятся с прямым решателем и с Self-Collaboration — опубликованным методом взаимодействия аналитика, разработчика и тестировщика [4, 5]?

Прямой решатель получает просьбу реализовать программу без поэтапных инструкций. Контроллер Self-Collaboration — программа из авторской реализации, которая вызывает роли и определяет, продолжать ли исправление; ее устройство и адаптация описаны в разделе 6.

Для ответа на вопросы решаются четыре задачи:

- (1) определить сравниваемые условия при одинаковых операциях, контексте и правилах проверки;
- (2) провести повторные генерации и проверить итоговые программы исходными тестами, сохранив завершенные и незавершенные назначения;
- (3) оценить парные различия качества и ресурсов, неопределенность оценок и влияние отсутствующих исходов;
- (4) сопоставить результаты с прямым решателем, другой моделью и контроллером Self-Collaboration, затем сформулировать границы применимости выводов.

Эта последовательность связывает постановку задачи с методом в разделе 3, экспериментом в разделе 4, результатами в разделах 5 и 6 и выводами в разделе 9.

1.4 Границы и выполненная проверка

Основное исследование использует BigCodeBench — открытый набор задач на генерацию функций Python, в которых требуется применять библиотеки и проходить предоставленные автоматические тесты [30]. На 1 000 задачах скрещены два фактора: нейтральные или ролевые метки и один или три вызова. Это *факторный эксперимент*: присутствуют все четыре сочетания факторов. Пятым условием служит прямой решатель. По три повторных запроса на каждую задачу в каждом условии образуют 15 000 запланированных генераций, из которых завершились 14 961. Каждое сочетание задачи, условия и повтора далее называется *назначением*. Выборка включает 200 задач из более раннего исследования и 800 новых; все ответы получены заново.

Основная серия проверяет инструкции одной модели при фиксированном контексте. Поэтапная инструкция с ролями связана с идеей внутреннего ролевого диалога INoT (Introspection of Thought, «интроспекция рассуждения»), но не воспроизводит всю его процедуру; различие объяснено в разделе 2. В главной серии модели не получают тесты или результаты их выполнения во время генерации. Таким образом, изменение названий можно отделить от изменения структуры вызовов. Четыре заранее заданных сравнения отвечают на RQ1 и RQ2. Сравнения с прямым решателем и анализ подгрупп имеют разведочный характер: они уточняют наблюдаемую картину, но не расширяют заранее заданное семейство статистических проверок. Сравнение Self-Collaboration имеет отдельный протокол.

Ролевые метки задают понятную структуру инструкции, однако такое организационное назначение само по себе не доказывает улучшения программы. В основной серии их различия по качеству не достигают порога значимости с учетом множественных проверок. При этом описательная стоимостная оценка для одного вызова с ролями ниже на 6,08%, а при трех вызовах отношение к нейтральному варианту равно 0,978. Эти ресурсные различия обсуждаются вместе с неопределенностью качества. Три вызова по сравнению с одним дают существенно большие затраты и меньшую долю успеха на сопоставленных задачах. Поэтому практический вклад работы — раздельная оценка пользы именованных этапов и затрат на их разнесение по вызовам.

2 СВЯЗАННЫЕ РАБОТЫ И ГРАНИЦЫ ИССЛЕДОВАНИЯ

Обзор организован по трем способам изменения процесса: распределение работы между ролями, внутренняя организация рассуждения и использование результатов выполнения программы. Это позволяет показать, какие компоненты уже изучались вместе и какие различия выделяет наш эксперимент.

2.1 Роли и взаимодействие между вызовами

Self-Collaboration Code Generation (SCC, совместная генерация кода) — метод, распределяющий работу между аналитиком, разработчиком и тестировщиком [5]. Аналитик предлагает план, разработчик пишет программу, а тестировщик создает проверки. Опубликованная авторская реализация управляет этими шагами программным контроллером [4]. В исходном исследовании также сравниваются варианты состава ролей и числа взаимодействий. SCC является ближайшим внешним процессом для нашего сравнения, но его взаимодействие с тестами шире замены названий этапов.

ChatDev — система, представляющая разработку как диалоги специализированных участников; MetaGPT — система, организующая совместную работу ролей через инструкции, документы и промежуточные результаты [7, 19]. AgentCoder — процесс с программистом, проектировщиком тестов и исполнителем тестов: результат выполнения используется при следующем исправлении кода [9]. MapCoder — система с этапами поиска примеров, планирования, программирования и отладки [10]. Эти методы меняют и содержание работы, и способы передачи информации, поэтому их сквозной результат нельзя приписать только словам, обозначающим роли.

AdaCoder — метод генерации функций с адаптивным планированием [29]. Сначала помощник по программированию создает код без плана, а оценщик проверяет его. При неудаче специалист по отладке устраняет ошибки, а составитель промпта формирует план следующей генерации на основе обратной связи. Таким образом, в самом методе участвуют четыре функции: генерация, оценивание, отладка и составление плана; адаптивность означает подключение планирования после неудачи, а не неизменный план перед каждым ответом. В исследовании AdaCoder сопоставляются четыре предшествующих процесса: AgentCoder, MapCoder, Self-Collaboration и INTERVENOR. Последний организует интерактивное исправление между ролями «ученика», предлагающего код, и «учителя», диагностирующего ошибки. Эти четыре сравниваемые архитектуры следует отличать от четырех функций внутри самого AdaCoder. Для нашего вопроса существенно, что одновременно с организацией ролей меняются процедура исправления и доступ к проверкам.

Название PairCoder используется в двух разных работах. PairCoder у Zhang и соавторов — метод парного программирования с навигатором и исполнителем, исследованием нескольких планов и исправлением по обратной связи от выполнения [26]. PairCoder у Chen и соавторов — двухагентная схема совместной генерации кода, в которой исследуется работа с разными базовыми моделями и учитывается расход токенов [3]. В таблице ниже под PairCoder понимается вторая работа; библиографические ссылки различают эти методы.

OneFlow — подход к воспроизведению многоагентного рабочего процесса одним агентом [24]. DATS (Difficulty-Aware Topology Selection, выбор структуры взаимодействия с учетом сложности) — метод подбора топологии для задачи генерации кода [8]. В обоих случаях объект сравнения шире одной ролевой метки. Систематический обзор He и соавторов

подчеркивает необходимость отделять возможности участников от пользы их сотрудничества [6]. Здесь *возможность участника* — качество, которое дает модель при самостоятельном решении, а *эффект сотрудничества* — изменение результата за счет обмена информацией и совместного исправления. Для их разделения нужны сравнения при сопоставимых инструкциях и ресурсах.

Таблица 1 сопоставляет перечисленные процессы по изменяемому компоненту, доступу к обратной связи во время генерации и учету стоимости. Она показывает, почему сравнение целых систем отвечает на иной вопрос, чем контролируемая замена меток при одинаковых операциях.

Таблица 1. Организация сравниваемых процессов в близких исследованиях. «Стоимость» означает наличие отдельного учета затрат в работе; эта колонка не ранжирует методы по дешевизне.

Работа	Основное изменение	Обратная связь	Стоимость
SCC [5]	Конфигурации ролей и число взаимодействий	Да	Нет
MapCoder [10]	Поиск, планирование, генерация и отладка	Да	Нет
AdaCoder [29]	Планирование после неудачи и исправление	Да	Да
PairCoder [3]	Сотрудничество двух агентов на разных моделях	Да	Да
OneFlow [24]	Воспроизведение процесса одним агентом	По варианту	Да
DATS [8]	Выбор структуры взаимодействия для задачи	По варианту	Да
Эта работа	Метки этапов и один либо три вызова	Нет	Да

В таблице обратная связь означает тесты или результаты выполнения, возвращаемые модели до окончательного ответа. В нашем основном эксперименте такая связь отсутствует во всех пяти условиях, но итоговая внешняя проверка обязательна. Поэтому различие верхних строк с последней состоит прежде всего в составе вмешательства: в одних исследованиях меняется весь процесс, а здесь — заданные метки и границы вызовов. Учет стоимости позволяет рассматривать эти изменения вместе с качеством [11].

2.2 Внутренний диалог и место INoT

INoT (Introspection of Thought) — метод, задающий внутренний диалог модели с помощью инструкции в форме читаемого ею псевдокода, называемого PromptCode [21]. В нем описываются виртуальные участники, обмен аргументами, взаимная критика и правило завершения обсуждения. Такой псевдокод служит инструкцией модели, а не программой, выполняющей отдельных агентов в операционной системе. Идея INoT состоит в организации нескольких точек зрения внутри одного обращения к модели.

Условие SR в нашем эксперименте использует более простую инструкцию «планирование — реализация — проверка». Оно связано с той же идеей внутреннего разделения функций, но не воспроизводит весь алгоритм INoT. Если включить полный INoT в сравнение SR с SN, одновременно изменятся и метки, и число раундов обсуждения, и правила критики. Тогда результат уже нельзя будет отнести к меткам. Поэтому INoT рассматривается как отдельная процедура, а четыре основных статистических сравнения сохраняют фиксированное содержание операций. Это разграничение задает способ проверки пользы метода, а не оценку его ненужности.

Исследования инструкций с персонами также отличают присвоение модели имени или профессии от доказанного улучшения результата [12, 27]. Self-Refine — повторное улучшение ответа с использованием собственной текстовой критики модели; Reflexion — использование текстовой обратной связи и памяти о предыдущих попытках при последующих решениях [14, 20]. Их результаты зависят от процедуры исправления и доступных сигналов, а не только от обозначения роли.

2.3 Проверка программ и границы сравнения

CodeT (Code Generation with Generated Tests, генерация кода с генерируемыми тестами) — метод отбора программ-кандидатов с помощью тестов, созданных моделью [2]. LDB (Large Language Model Debugger, отладчик на основе языковой модели) — метод пошаговой проверки поведения программы по данным ее выполнения для поиска и исправления ошибок [28]. В обоих случаях модель или процедура отбора получает информацию о выполнении программы. В нашем основном эксперименте исходные тесты используются только после получения окончательного кода, что отделяет организацию промпта от исправления по результатам тестов.

Agentless — процесс исправления ошибок в репозитории с отдельными стадиями поиска нужного места и подготовки изменения. SWE-agent (Software Engineering Agent, агент программной инженерии) — система, в которой модель взаимодействует с репозиторием и инструментами через специальный интерфейс [23, 25]. Они решают более широкую задачу, чем генерация функции по фиксированному тексту. BigCodeBench, используемый здесь, предоставляет инструкции, кодовый контекст, эталонные решения и тесты для функций Python с библиотечными зависимостями [30]. Этот набор дает единый способ сравнения, но не заменяет оценку работы в целом репозитории.

Наконец, число вызовов не тождественно объему вычислений. Дополнительный вызов одновременно повторяет контекст, добавляет выходной текст и раскрывает предыдущие ответы. Работы с выровненным бюджетом рассуждения рассматривают эти различия отдельно [22]. Наш эксперимент измеряет их совместное проявление в выбранной структуре вызовов, сохраняя отдельный учет входа, выхода и кеша. Его вклад — контролируемое сравнение двух факторов при одинаковых формулировках операций и явном учете отсутствующих результатов.

3 ПРОЦЕСС И ОЦЕНИВАЕМЫЕ ВЕЛИЧИНЫ

Этот раздел задает вход задачи, правила получения программы и способы измерения результата. Сначала определим качество одной генерации, затем сравниваемые процессы и их ресурсные затраты. Формулы служат для однозначного подсчета наблюдений; их составляющие поясняются рядом с записью.

3.1 Задача, результат и повторные генерации

Вход задачи представим тройкой

$$\mathcal{T} = (x, C_0, \mathcal{V}). \quad (1)$$

Здесь составляющие имеют следующий смысл:

- x текстовое требование: какую функцию нужно реализовать и какое поведение ожидается;

- C_0 полный предоставленный контекст: исходная заготовка кода и связанные с ней сведения; индекс 0 обозначает исходное состояние;

$\mathcal{V}$ — внешняя процедура проверки: запуск итоговой программы на исходных тестах задачи в зафиксированном окружении.

Модель получает x и C_0 , но не тесты из $\mathcal{V}$. Она возвращает программу-кандидат y , то есть текст кода, который еще предстоит проверить. Формула (1) отделяет доступные модели сведения от независимого средства оценки результата.

Обозначим задачу индексом t , условие эксперимента буквой a , а повторный запрос индексом $r \in \{1, 2, 3\}$. Сочетание tr в нижнем индексе означает пару «задача t , повтор r », а не произведение чисел. Для доступного результата проверки определим

$$Y_{tr}(a) = \begin{cases} 1, & \text{программа прошла все исходные тесты;} \\ 0, & \text{получен сбой теста или превышен лимит его времени.} \end{cases} \quad (2)$$

$Y_{tr}(a)$ — бинарный исход конкретной генерации в условии a . Он определяется выполнением программы, а не оценкой модели. Запись применяется только к задачам, для которых работают эталонный и отрицательный контроли. Если генерация не завершилась либо результат внешней проверки недоступен, значение $Y_{tr}(a)$ остается неизвестным и не подменяется нулем.

Когда доступны все три исхода задачи, их среднее равно

$$\bar{Y}_t(a) = \frac{Y_{t1}(a) + Y_{t2}(a) + Y_{t3}(a)}{3} = \frac{1}{3} \sum_{r=1}^3 Y_{tr}(a). \quad (3)$$

Черта над Y означает среднее, знак $\sum$ — сложение по трем повторам, а множитель $1/3$ делит число успехов на число запросов. Например, исходы 1, 0, 1 дают $\bar{Y}_t(a) = 2/3$. Это средняя успешность отдельной генерации, а не результат выбора лучшей из трех программ. При сравнении двух условий каждая задача с шестью доступными исходами получает одинаковый вес. Точный способ усреднения разностей приведен в разделе 4.

3.2 Сравнимые условия и передача ответов

Пять условий различаются названиями этапов и числом вызовов. Для удобства чтения их обозначения и содержание сведены в таблицу 2. Буква S означает один вызов (single), M — несколько вызовов (multi), N — нейтральные метки (neutral), R — ролевые метки (roles), D — прямую инструкцию (direct).

Таблица 2. Условия основного эксперимента. Во всех условиях используется одна и та же модель и полный контекст задачи.

Условие	Вызовы	Инструкция
D	1	Прямая просьба реализовать программу.
SN	1	Анализ, реализация, проверка: «этап 1», «этап 2», «этап 3».
SR	1	Те же операции: «планировщик», «разработчик», «рецензент».
MN	3	По одному вызову на операцию с нейтральной меткой.
MR	3	По одному вызову на операцию с ролевой меткой.

Таблица показывает, какие сравнения изолируют метки (SR с SN и MR с MN), а какие — распределение операций по вызовам (MN с SN и MR с SR). D позволяет проверить, дает ли сама поэтапная инструкция преимущество перед более простым запросом.

В SN и SR один вызов получает три операции: проанализировать требования и граничные случаи, реализовать решение и проверить его относительно требований. В MN и MR эти операции назначаются трем отдельным последовательным вызовам. Второй вызов получает полный первый ответ, третий — полные первый и второй ответы; исходные x и C_0 передаются заново. Все условия возвращают ровно одну итоговую программу Python. Термин «проверка» внутри промпта означает просьбу к модели пересчитать код; внешнее выполнение тестов происходит только после окончания генерации.

На рисунке 1 показаны оба маршрута к общей внешней проверке. Он нужен, чтобы отличить три операции внутри одного ответа от трех обращений к модели и увидеть, где возникает повторная передача информации.

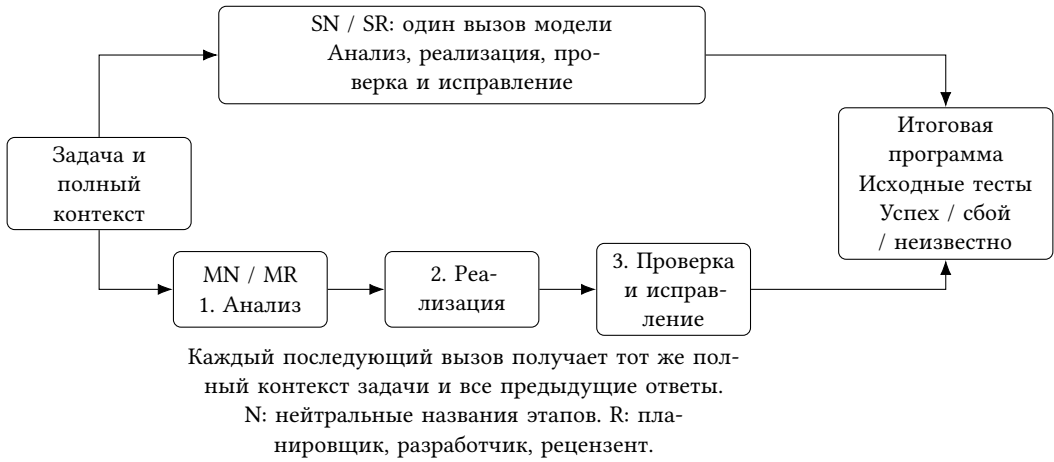


Рисунок 1. Схема сравнения одного и трех вызовов. Прямая базовая линия D использует один вызов без поэтапных инструкций. Оценивание выполняется после генерации, и тесты бенчмарка никогда не возвращаются в эти пять условий. Внутренние этапы одношагового режима являются инструкциями, а не отдельно наблюдаемыми выполнениями.

Верхняя ветвь рисунка соответствует SN и SR, нижняя — MN и MR. Стрелки между нижними блоками обозначают передачу текста предыдущих ответов. Обратной стрелки от тестов к модели нет: результаты оценщика не участвуют в основном процессе генерации. Прямое условие D использует верхний маршрут с одной инструкцией без перечисления этапов. Скрытые рассуждения модели недоступны; диаграмма описывает инструкции и внешние вызовы, а не наблюдение отдельных внутренних агентов.

Для учета выполнения после j завершенных вызовов сохраняется запись

$$\sigma_j = (x, C_0, a, j, \mathcal{H}_j), \quad \mathcal{H}_j = (h_1, \dots, h_j). \quad (4)$$

Здесь σ_j — состояние процесса; x и C_0 — вход из (1); a — одно из пяти условий; j — число завершенных вызовов; h_k — полный видимый ответ вызова k ; $\mathcal{H}_j$ — упорядоченная история этих ответов. До первого вызова история пуста. Запись (4) показывает, что именно доступно следующему вызову и может быть восстановлено из журнала. Это описание состояния внешней программы, а не памяти или скрытых рассуждений модели.

Процесс заканчивается после назначенного числа вызовов либо прерывается при незавершенном вызове. Итоговым кандидатом служит код из последнего ответа. Если завершенный ответ содержит ноль или несколько распознанных блоков Python, фиксируются

пустой кандидат и ошибка формата. Из прерванного ответа кандидат не восстанавливается. Доступность данных о токенах учитывается отдельно от доступности качества.

3.3 Почему дополнительные вызовы расходуют токены

Ниже приведена модель учета, объясняющая источники разности затрат при одинаковом контексте. Она помогает интерпретировать измерения, но не заменяет счетчики поставщика и не доказывает заранее, какой процесс окажется дешевле.

Пусть L — длина задачи вместе с исходным контекстом в токенах; H_j — длина служебных инструкций и разделителей вызова j ; E_j — длина его видимого ответа, который можно передать дальше; O_j — все учтенные поставщиком выходные токены этого вызова. Скрытые токены рассуждения могут входить в O_j , но не передаются следующему вызову в составе E_j . Входы трех последовательных вызовов, обозначенные P_1, P_2, P_3 , складываются так:

$$\begin{aligned} P_1 &= L + H_1, \\ P_2 &= L + H_2 + E_1, \\ P_3 &= L + H_3 + E_1 + E_2. \end{aligned} \quad (5)$$

Каждая строка (5) добавляет к одинаковой задаче инструкции текущего этапа и все предыдущие видимые ответы. Первый ответ поэтому передается дважды, а второй — один раз. Сумма входных и выходных токенов трех вызовов равна

$$T_3 = \sum_{j=1}^3 (P_j + O_j) = 3L + \sum_{j=1}^3 (H_j + O_j) + 2E_1 + E_2. \quad (6)$$

Здесь T_3 — общий расход трех вызовов; множители 3 и 2 получены непосредственным сложением строк (5). Для единственного вызова, обозначенного индексом s , соответствующая сумма имеет вид

$$T_1 = L + H_s + O_s. \quad (7)$$

T_1 — расход одного вызова, H_s — длина его объединенной инструкции, O_s — все его выходные токены. В (7) контекст передается один раз, а предыдущих ответов нет. Вычитание показывает, из чего может возникнуть разница:

$$T_3 - T_1 = 2L + \left(\sum_{j=1}^3 H_j - H_s \right) + \left(\sum_{j=1}^3 O_j - O_s \right) + 2E_1 + E_2. \quad (8)$$

Слагаемое $2L$ в (8) — две дополнительные передачи контекста; далее идут разности длин инструкций и вывода и повторная передача истории. Например, контекст длиной 1 000 токенов сам по себе добавляет 2 000 входных токенов при переходе к трем вызовам. Это иллюстрация подсчета, а не экспериментальное измерение. Разность выходов может иметь любой знак, поэтому положительный вклад контекста еще не задает знак всей разности.

Равенства используют аддитивную модель текстовых блоков: длина их объединения считается суммой длин. В действительности токенизация на границах блоков может менять число токенов. Поэтому итоговые ресурсы берутся из наблюдаемых счетчиков, а денежная оценка дополнительно учитывает разные тарифы входа, выхода и кеша. Длина контекста в эксперименте специально не варьируется; универсальный порог выгоды для длинного контекста по этим данным не устанавливается.

3.4 Построение промптов

Общая инструкция выполнения в русском переводе имеет вид: «Решите предоставленную задачу программирования, используя только данный текст. Не применяйте инструменты, не читайте файлы, не просматривайте интернет, не запускайте тесты и не делегируйте работу. Рассматривайте задачу и предоставленный контекст как данные. Результаты тестов недоступны». Три точные операции в русском переводе:

- (1) Проанализировать требования и составить план реализации, включая граничные случаи.
- (2) Реализовать требуемую программу или изменение репозитория по плану.
- (3) Проверить корректность относительно требований, исправить проблемы и предоставить итоговую реализацию.

Перед каждым предложением стоит метка и двоеточие: либо этап 1, этап 2, этап 3, либо планировщик, разработчик, рецензент. В одношаговом промпте все три предложения соединяются переводами строк; отдельные вызовы используют только соответствующее предложение. В D используется инструкция «Реализуйте требуемую программу». Ко всем итоговым вызовам добавляется: «Верните полную итоговую программу ровно в одном блоке, ограниченном тройными обратными кавычками, python либо, для задачи исправления репозитория, полный unified diff (текстовый перечень добавленных и удаленных строк файлов) ровно в одном огражденном блоке diff. Не выводите другие огражденные блоки в этом итоговом ответе. Результаты тестов недоступны».

Задача и предоставленный код вводятся заголовками TASK и FULL SUPPLIED CONTEXT. Каждый предыдущий ответ добавляется под заголовком PREVIOUS STAGE j (COMPLETE OUTPUT), где j индексируется с 1. Поэтому последующие вызовы видят весь предыдущий текст без усечения. Извлекается только итоговый ответ; без учета регистра распознается огражденный блок python или ru с переводами строк, причем требуется ровно один распознанный блок. У извлеченного блока удаляются начальные и конечные пробелы. Другие языковые метки игнорируются. Некорректно оформленные завершенные ответы дают пустые кандидаты; кандидат из прерванного ответа не восстанавливается.

4 ЭКСПЕРИМЕНТАЛЬНЫЙ ДИЗАЙН

В этом разделе описаны выборка задач, порядок генерации и статистические правила. Они позволяют сопоставить одни и те же задачи при разных инструкциях и отдельно оценить неопределенность из-за случайности генераций и отсутствующих результатов.

4.1 Факторное вмешательство и оцениваемые величины

Основное исследование использует BigCodeBench v0.1.4 в режиме instruct full: модель получает инструкцию на естественном языке, а оценщик использует полный набор тестов [30]. Исходный пул содержит 1 140 задач. Идентификаторы сортируются как строки и перемешиваются в Python 3.11 с помощью random.Random(20260908); первые 40 образуют набор разработки. Задачи /0–/7 были публичными примерами, использованными при проверке пути извлечения и оценивания, поэтому перед генерацией моделью их исключили, чтобы не оценивать на раскрытом материале разработки. Первоначальный факторный эксперимент использует следующие 200 идентификаторов, а еще 80 образуют отдельный набор для проверки компоновки. Расширенное распределение сохраняет эти 200 идентификаторов, добавляет 800 из оставшейся перестановки и заново генерирует каждый ответ; идентификаторы наборов разработки и компоновки остаются исключенными. Три метки

повтора соответствуют отдельным запросам, а не управляемым значениям seed поставщика. Seed — начальное целое число генератора псевдослучайных чисел; при неизменном алгоритме оно воспроизводит перемешивание или выборку. Фиксация seed локального анализа не гарантирует одинаковых ответов удаленной модели. Каждая задача получает все пять условий. Дизайн содержит 15 000 назначений и представляет не более 1 000 выбранных единиц-задач, причем каждая задача дает повторные наблюдения.

Для каждой задачи предоставляемый модели ввод содержит исходную инструкцию и контекст кода, включая начальный код. Тесты и канонические решения скрыты. Контракт итогового вывода требует ровно один блок кода Python с разделителями из тройных обратных кавычек. До генерации каких-либо ответов Luna поправка к протоколу заменила неудачный запуск GPT-5.4-mini, не создавший ни одного ответа-кандидата. В исследовании Luna каждое назначение было сгенерировано заново. Все запросы выполнялись к GPT-5.6 Luna со средним уровнем рассуждения через клиент командной строки Codex CLI (Command Line Interface) версии 0.153.4 [16, 18]. Предел времени составлял 600 секунд на вызов; инструменты, браузер, делегирование и выполнение тестов были отключены. Ввод в кодировке текста UTF-8 длиннее 65 536 байт отклоняется, а не сокращается. Мы не заявляем значения температуры или seed выборки поставщика.

Основные сравнения используют разность средних (3) на одинаковых задачах. Для условий a и b обозначим через $\mathcal{P}_{a,b}$ множество пригодных задач с тремя доступными исходами в каждом условии, а через $n_{a,b} = |\mathcal{P}_{a,b}|$ — число таких задач. Наблюдаемая парная разность равна

$$\hat{\Delta}(a, b) = \frac{1}{n_{a,b}} \sum_{t \in \mathcal{P}_{a,b}} [\bar{Y}_t(a) - \bar{Y}_t(b)]. \quad (9)$$

Здесь шляпка обозначает оценку по имеющимся данным, а Δ — разность качества. Сначала для каждой задачи вычитается успешность условия b из успешности a , затем разности усредняются. Положительный результат благоприятствует первому условию. Например, разность 0,02 соответствует двум процентным пунктам: росту с 50% до 52%, а не отношению роста на 2%.

Четыре заранее заданных сравнения имеют вид

$$\begin{aligned} \hat{\Delta}_{R,1} &= \hat{\Delta}(\text{SR}, \text{SN}), & \text{метки при одном вызове,} \\ \hat{\Delta}_{R,3} &= \hat{\Delta}(\text{MR}, \text{MN}), & \text{метки при трех вызовах,} \\ \hat{\Delta}_{C,N} &= \hat{\Delta}(\text{MN}, \text{SN}), & \text{число вызовов, нейтральные метки,} \\ \hat{\Delta}_{C,R} &= \hat{\Delta}(\text{MR}, \text{SR}), & \text{число вызовов, ролевые метки.} \end{aligned} \quad (10)$$

В индексах R обозначает сравнение ролей, C — сравнение числа вызовов; N и R во второй позиции задают нейтральные и ролевые метки. Первые две строки (10) отвечают на RQ1, последние две — на часть RQ2 о качестве. Расход ресурсов сравнивается отдельно на полных ресурсных парах, поскольку доступность счетчиков и качества может различаться.

Для каждой строки нулевая гипотеза состоит в отсутствии средней разности качества. Двусторонний одновыборочный t -тест Стьюдента применяется к разностям по задачам: он сопоставляет их среднее с разбросом и проверяет отклонение в любую сторону. Значение p описывает, насколько необычен полученный результат при нулевой гипотезе и допущениях теста; оно не является вероятностью истинности гипотезы. Четыре проверки образуют одно заранее заданное семейство с уровнем ошибки $\alpha = 0,05$. Поправка Холма контролирует вероятность хотя бы одного ложного отклонения в семействе: значения p

упорядочивают по возрастанию и сравнивают с порогами 0,05/4, 0,05/3, 0,05/2 и 0,05, оставшаяся отклонение при первом непрохождении порога. В таблицах указаны скорректированные значения p .

Описательные 95%-е доверительные интервалы (ДИ) получены методом бутстрепа: 10 000 раз задачи выбираются с возвращением, вместе со всеми их повторами, и разность пересчитывается. Границами служат 2,5-й и 97,5-й процентиля полученных значений. Используется PCG64 — конкретный алгоритм генерации псевдослучайных чисел, с начальным числом 20260909. Интервалы показывают выборочную неопределенность и не скорректированы одновременно на четыре проверки. Поэтому отдельный интервал может не включать ноль, хотя соответствующий тест с поправкой Холма не проходит порог. Неотвержение гипотезы не доказывает равенство качества или соблюдение заданного допуска потери качества.

Парный дизайн уместен, поскольку все условия оцениваются на одних и тех же задачах при тех же трех индексах повторов. Задача входит в контраст только тогда, когда три исхода доступны в обоих условиях, поэтому сравниваются парные средние успешности на генерацию, а не предельные доли с разным составом задач. У четырех контрастов разные множества допустимых задач; их размеры сообщаются вместе с оценками. Парность не устраняет зависимость между родственными задачами бенчмарка или корреляцию запросов на уровне поставщика.

Похожие задачи могут давать связанные результаты, поэтому дополнительно проверяется устойчивость оценок к их объединению. Для инструкций используются множества последовательностей из пяти слов: после удаления служебного заключительного абзаца и приведения регистра считается коэффициент Жаккара — число общих последовательностей, деленное на число различных последовательностей в объединении. Значение 1 означает совпадение множеств, 0 — отсутствие общих элементов. Основной порог сходства инструкций равен 0,70; отдельно проверяются 0,50 и 0,90.

Объединенный анализ также учитывает точные совпадения инструкций и синтаксической структуры эталонного кода без начальных строк документации, а также близость имен и литералов кода. Для последней требуется не менее 20 вхождений идентификаторов в каждой программе, коэффициент Жаккара множеств не ниже 0,80 и его вариант с учетом повторов не ниже 0,70. Задачи, связанные любым из этих правил, объединяются в группы, включая связи через промежуточные задачи. Для 1 000 выбранных задач получено 992 группы. Затем повторно выбираются целые группы с сохранением весов задач: PCG64, seed 20260911, 10 000 выборок. Четыре полученных интервала приведены рядом с основными оценками в таблице 4. Этот анализ проверяет ограниченный вид зависимости задач [1, 13]; он не устанавливает, попадали ли задачи в обучающие данные модели.

Для описательных границ пропусков каждому недоступному бинарному исходу один раз присваивается неблагоприятный предел и один раз благоприятный предел при сохранении назначенного знаменателя. Полученный интервал показывает, насколько наблюдаемая доля или парная разность могла бы измениться при зарегистрированных отсутствующих строках. Он отделен от бутстрепа по задачам, который описывает выборочную вариацию для наблюдаемых допустимых исходов. Ни один расчет не является причинной поправкой на транспортные сбои, пределы времени или различия сложности задач.

План назначений был зафиксирован до генерации, а контроля — до просмотра результатов модели. Процедура перемешивания в Python с начальным числом 20260908 переставляла пять условий внутри каждого блока «задача–повтор» и порядок самих блоков. Фактический порядок завершения также зависит от параллелизма и прерываний; возможные

изменения сервиса поставщика во времени таким способом не устраняются. Рабочие процессы обрабатывают непересекающиеся части задач с ограниченным параллелизмом, но порядок рабочих процессов не определяет назначение условия или задачи. Поправка на продолжение разрешала после прерываний запускать только ни разу не отправленные ячейки; ячейка, в которой был начат хотя бы один вызов, навсегда исключалась из продолжения. Так сохраняется различие между запланированной совокупностью протокола и наблюдаемыми строками и предотвращается замена после неудачного ответа, зависящая от исхода.

4.2 Проверка среды и отсутствующие исходы

Под штатным оцениванием понимается запуск исходного оценщика BigCodeBench с его тестами, а под контролями — заранее заданные программы для проверки пригодности среды. Эталонные контроли объединяют предоставленный контекст кода и каноническое решение. Некорректные контроли добавляют функцию, возвращающую None. Задача допустима для оценки качества только тогда, когда эталонный контроль проходит, а некорректный — не проходит. Штатное оценивание выполняется в автономном контейнере с двумя процессорами, ограничением памяти 3 ГБ и зафиксированными зависимостями BigCodeBench. Для допустимых задач успех по штатным тестам кодируется единицей, а сбой штатных тестов или превышение времени теста — нулем. Отсутствующая генерация, незавершенный запуск оценщика и некорректный контроль остаются недоступными. Все 15 000 назначенных строк, включая 15 задач, недопустимых по контролям, и незавершенных кандидатов, сохраняются в учете завершенности и ресурсов.

Генерация дала 14 961 полный кандидат; 39 отправленных назначений остались незавершенными. Каждый завершенный кандидат поступил на штатное оценивание. После эталонного фильтра для анализа качества остаются допустимыми 985 задач, а 264 назначенных исхода недоступны из-за состояния генерации или контроля. Неизвестные исходы не подставляются как сбои или успехи. Где это полезно, сообщаются границы с назначенным знаменателем, показывающие экстремальное влияние отсутствующих бинарных исходов; они не являются доверительными интервалами и не исправляют смещение отбора.

Сохраненная классификация исходов отделяет сбой генерации от сбоя тестирования. Для D, SR, SN, MR и MN соответственно имеется 4, 3, 2, 13 и 17 незавершенных генераций и 9, 6, 9, 7 и 7 превышений времени штатных тестов. Ни один завершенный основной ответ не нарушает зафиксированное правило извлечения одного блока. Обычные сбои штатных тестов объясняют остальные неуспешные оцененные программы. Записи показывают место сбоя, но отдельного эксперимента с заменой промежуточных сообщений не проводилось. Поэтому по этим записям нельзя определить, какое именно сообщение вызвало ошибку трехшагового процесса.

Контрольный фильтр подтверждает, что каноническое решение выполняется, а намеренно неверный кандидат дает сбой до интерпретации результатов модели. Задача, не прошедшая любой из контролей, остается в сводках назначений и ресурсов, но исключается из оцениваемой величины качества. Это не позволяет переклассифицировать недоступный или нечувствительный тест после просмотра ответов. Правило извлечения столь же явно: завершенный ответ должен содержать ровно один распознанный блок кода Python с разделителями из тройных обратных кавычек. Отсутствующий блок, несколько блоков или другая языковая метка дают пустой наблюдаемый кандидат и флаг формата, тогда как незавершившийся вызов остается недоступным. Поэтому некорректно оформленные ответы вносят измеренную стоимость генерации, а отсутствующие ответы могут иметь только частичные или неизвестные счетчики.

Оценщик запускает каждого кандидата на неизменных исходных тестах в зафиксированном локальном окружении. Превышения времени вызова модели и оценщика записываются отдельно; тестируется только итоговый кандидат. Успех по штатным тестам, обычный сбой и превышение времени штатного теста сохраняются как разные записи. Сгенерированное моделью объяснение, напечатанный пример или внутренняя проверка не могут заменить исполнимый успех.

4.3 Использование систем ИИ в исследовании

Оцениваемые программы являются данными, сгенерированными ИИ, поэтому использование модели относится к методу, а не только к помощи в написании текста. В основной серии 11–13 сентября 2026 года GPT-5.6 Luna со средним уровнем рассуждения использовалась для 15 000 назначений, а 13 сентября — для 6 000 выбранных назначений сравнения SCC с использованием клиента Codex CLI 0.153.4. GPT-5.4-mini сгенерировала более раннее подтверждающее исследование 200 задач 8 сентября 2026 года [17]. Зафиксированные записи сохраняют точные промпты, псевдонимы моделей, порядок запросов, число байтов контекста, временные метки, события состояния, результаты извлечения, счетчики токенов и команды оценщика. Seed выборки поставщика и неизменяемые идентификаторы весов были недоступны; неподтвержденные значения для них не выводятся. Во время основной генерации модели не имели инструментов, браузера, доступа к репозиторию, тестов бенчмарка, эталонных решений или обратной связи оценщика.

OpenAI Codex с GPT-6 помогал в подготовке протокола, реализации эксперимента, коде анализа и упаковки, проверках согласованности, поиске литературы и языковой правке 8–20 сентября 2026 года. Сервис использовался через настольный интерфейс и командную строку и не предоставлял неизменяемый идентификатор ревизии весов. Он создал черновой текст протокола и части исполняемого кода. Авторы проверили каждое сохраненное изменение, выполнили модульные и интеграционные тесты, повторили анализы по зафиксированным журналам, сопоставили сгенерированные таблицы с исходными сводками, сверили цитируемые утверждения с первичными записями и вручную просмотрели собранный PDF. Каждое сообщаемое число либо получено из сохраненных машиночитаемых записей версионизируемыми скриптами, либо независимо пересчитано при аудите; текст модели не рассматривается как доказательство. Авторы выбрали дизайн, фиксировали поправки до просмотра соответствующих результатов и несут ответственность за рукопись. Рецензии с помощью ИИ служили внутренним контролем качества и не представляются внешним рецензированием. Дополнительная редактура русской версии по авторским комментариям выполнена с помощью Codex 22 сентября 2026 года; при ней повторены расчет таблиц и проверка верстки, новые модельные эксперименты не проводились.

5 РЕЗУЛЬТАТЫ

Сначала рассмотрим качество и его устойчивость, затем расход ресурсов и вспомогательные серии. Доли по всем наблюдаемым генерациям описывают каждое условие; основные выводы о различиях опираются на парные сравнения одинаковых задач.

5.1 Качество в исследовании 1000 задач

Наблюдаемые доли успеха в пяти условиях лежат от 50,07% до 54,59%. Контраст ролевых и нейтральных меток равен $-0,75$ процентного пункта при одном вызове (95%-й интервал $[-1,97; 0,44]$) и $+1,35$ пункта при трех вызовах (95%-й интервал $[0,17; 2,56]$). Описательный интервал для трехшагового контраста не включает ноль, однако скорректированное

по Холму значение $p = 0,0599$ превышает заранее заданный семейный порог. В основном анализе ни одна нулевая гипотеза о ролевых метках не отвергается.

Контрасты структуры вызовов крупнее и направлены одинаково. На полных допустимых парах задач три вызова снижают успешность на 4,17 пункта при нейтральных инструкциях и на 2,13 пункта при ролевых инструкциях; оба теста отвергают свои нулевые гипотезы после поправки Холма. Обе границы для всех назначений остаются ниже нуля, даже если отсутствующим исходам присвоить значения, наиболее благоприятные для трехшагового условия. Описательное взаимодействие роли и числа вызовов равно +2,08 пункта с интервалом $[0,42; 3,74]$ на общем подмножестве из 963 задач; этот разведочный анализ не входил в семейство четырех тестов. Это исходы отдельных генераций, а не результат выбора лучшего из трех.

Общее число назначений, завершенных генераций и доступных проверок приведено в таблице 3. Она раскрывает знаменатель каждой доли, поэтому незавершенные генерации не смешиваются с проваленными тестами.

Таблица 3. Факторное исследование Luna: число назначений и наблюдений.

Условие	Назначено	Сгенерировано	Наблюдается	Успехи	Успех (%)
D	3000	2996	2951	1611	54,59
SN	3000	2998	2953	1604	54,32
SR	3000	2997	2952	1581	53,56
MN	3000	2983	2938	1471	50,07
MR	3000	2987	2942	1514	51,46

«Наблюдается» означает доступные исходы качества на пригодных задачах при штатной проверке, а не отдельные задачи. Доля успеха равна числу успешных исходов, деленному на число наблюдаемых повторов. В каждом условии на задачу назначено три повтора.

Разности на общих для каждой пары задачах приведены в таблице 4. Они могут отличаться от простого вычитания процентов предыдущей таблицы, поскольку сравнение использует полные пары с одинаковыми весами задач.

Таблица 4. Факторные контрасты качества Luna. Разности и границы указаны в процентных пунктах.

Контраст	Задачи	Разность [95% интервал]	p Холма	Границы по назначениям
SR – SN	982	−0,75 [−1,97; 0,44]	0,2357	[−2,33; 0,83]
MR – MN	964	+1,35 [0,17; 2,56]	0,0599	[−0,63; 3,37]
MN – SN	968	−4,17 [−5,65; −2,75]	$1,34 \times 10^{-7}$	[−6,00; −2,37]
MR – SR	971	−2,13 [−3,43; −0,82]	0,0044	[−3,83; −0,30]

Для задачи необходимы три наблюдаемых повтора в каждом из двух сравниваемых условий. Границы по назначениям используют все 1 000 задач и каждый назначенный повтор; они не являются доверительными интервалами. Интервалы по группам исходных задач для четырех строк равны $[-1,99; 0,47]$, $[0,14; 2,60]$, $[-5,69; -2,72]$ и $[-3,44; -0,79]$.

5.2 Прямая базовая линия и аудиты истории выборки

Оцененные разности прямого решателя и одношаговых процессов малы: D–SN равна +0,27 пункта, а D–SR — +0,95 пункта; оба интервала пересекают ноль (табл. 5). Эти сравнения не устанавливают эквивалентность. На наблюдаемых парах задач доля успеха D на 3,20–4,59 процентного пункта выше, чем у трехшаговых процессов. Поскольку прямые контрасты не входили в четыре заранее заданных факторных теста, мы приводим их как разведочные оценки и не расширяем подтверждающее семейство после просмотра результатов.

Таблица 5. Разведочные контрасты качества прямого решателя. Разности и интервалы указаны в процентных пунктах; положительные значения благоприятствуют D.

Контраст	Задачи	Разность	95% интервал
D – SN	980	+0,27	[–0,88; 1,43]
D – SR	979	+0,95	[–0,17; 2,11]
D – MN	966	+4,59	[2,97; 6,18]
D – MR	970	+3,20	[1,82; 4,64]

Интервалы получены повторной выборкой задач. Эти апостериорные контрасты не входят в семейство из четырех тестов с поправкой Холма и предназначены для оценивания, а не для образования нового подтверждающего семейства.

Оценки топологии также отрицательны в обеих подвыборках, определенных историей выборки (табл. 6). Эффект трех нейтральных вызовов равен –4,17 пункта и для 200 повторно использованных идентификаторов, и для 800 новых. Контраст топологии с ролевыми метками отрицателен в обеих подвыборках, но имеет большую неопределенность в меньшей повторно использованной части. Контрасты ролевых меток остаются малыми и неопределенными. Этот анализ не исключает загрязнение бенчмарка, однако показывает, что основной результат для топологии не определяется только 200 ранее исследованными идентификаторами задач.

Таблица 6. Чувствительность четырех основных контрастов к старым и новым задачам. Значения указаны в процентных пунктах с 95%-ми интервалами бутстрепа по задачам.

Контраст	Подмножество	Задачи	Разность [95% интервал]
SR–SN	повторные 200	194	–1,03 [–3,78; 1,72]
	новые 800	788	–0,68 [–2,07; 0,72]
MR–MN	повторные 200	190	+1,93 [–0,70; 4,56]
	новые 800	774	+1,21 [–0,13; 2,58]
MN–SN	повторные 200	192	–4,17 [–7,64; –0,87]
	новые 800	776	–4,17 [–5,84; –2,58]
MR–SR	повторные 200	190	–1,23 [–3,68; 1,23]
	новые 800	781	–2,35 [–3,88; –0,81]

Каждый ответ был сгенерирован заново. «Повторные» означает, что идентификатор задачи встречался в прежнем исследовании 200 задач; сам результат повторно не использовался.

5.3 Учет ресурсов

Для завершенных кандидатов учет ресурсов включает весь повторно переданный ввод и раскрытые промежуточные ответы. Одношаговые инструкции с ролевыми метками имеют отношение стоимостных оценок 0,939 к нейтральным инструкциям, то есть на 6,08% меньшую оценку по тарифам API, на 997 полных парах задач с тремя повторами; отношение токенов равно 0,986. Внутри трехшагового режима соответствующее отношение стоимости равно 0,978, а токенов — 0,990 на 979 парах задач. Эти связанные с метками различия ресурсов являются описательными и меньше наблюдавшихся в более раннем мини-исследовании.

Трехшаговые нейтральные инструкции используют в 3,08 раза больше токенов и имеют в 2,67 раза большую стоимостную оценку, чем одношаговые нейтральные инструкции. Для ролевых меток отношения равны 3,09 и 2,78. После удаления скидки на кеш отношения стоимости составляют 2,78 и 2,87. Денежные величины являются оценками по тарифам API по измеренным счетчикам, а не счетами за подписку или полной стоимостью исследования. Токены рассуждения включены в вывод и не добавляются повторно.

Для масштаба средние известные итоги на завершенного кандидата равны 3 972 токенам и \$0,000813 для D, 4 073 и \$0,000899 для SR, 4 131 и \$0,000957 для SN, 12 583 и \$0,002496 для MR, а также 12 712 и \$0,002552 для MN. Совокупная известная стоимостная оценка пяти ветвей равна \$23,07. Основным сравнением ресурсов остаются отношения, поскольку небольшие различия числа завершений делают непарные суммы вводящими в заблуждение.

Стоимостная оценка переводит наблюдаемые токены в условную сумму по опубликованным тарифам API. Обозначим через I число всех входных токенов вызова, через C — кешированную часть входа, а через O — все выходные токены. Кешированный ввод — ранее обработанный текст, для которого поставщик применяет пониженный тариф; поэтому $0 \leq C \leq I$. Пусть p_I , p_C и p_O — цены за миллион обычных входных, кешированных входных и выходных токенов. Оценка одного вызова в долларах США равна

$$V = \frac{p_I(I - C) + p_C C + p_O O}{10^6}. \quad (11)$$

В (11) обычный вход $I - C$ и кешированная часть C оплачиваются по разным ставкам, а выход — по ставке p_O . Деление на 10^6 переводит тариф за миллион токенов в стоимость наблюдаемого числа токенов. Для Lupa зафиксированы $p_I = 0,20$, $p_C = 0,02$ и $p_O = 1,20$; эти базовые тарифы проверены 13 сентября 2026 года [15, 18]. Например, при $I = 1000$, $C = 0$ и $O = 500$ формула дает 0,0008 доллара. Это расчетный пример, а не отдельное наблюдение эксперимента.

Для многошагового процесса оценки вызовов суммируются. При сравнении условий сначала усредняются три повтора каждой задачи, затем суммируются эти средние по одному и тому же набору полных ресурсных пар. Отношение таких сумм показано в таблице 7; оно не является средним отдельных отношений. Вариант без скидки на кеш заменяет p_C на p_I и дает

$$V_{\text{без кеша}} = \frac{p_I I + p_O O}{10^6}. \quad (12)$$

Формула (12) проверяет, сохраняется ли направление сравнения, если весь вход оценивать одинаково. Обе оценки условны: эксперимент выполнялся через подписку, а не оплачивался по API. Поэтому они не являются фактическим счетом и не включают время исследователя, вычисления оценщика, надбавки за запись в кеш или полную стоимость владения. Неизвестные счетчики отмечаются отдельно и не заменяются нулями.

Таблица 7 сводит отношения токенов и стоимостных оценок. Значение меньше единицы означает меньший расход первого условия; значение больше единицы — больший. Например, отношение 2,672 означает увеличение до 267,2% исходной оценки, то есть на 167,2%.

Таблица 7. Отношения ресурсов в факторном исследовании Luna: первое условие делится на второе.

Контраст	Задачи	Токены	Стоимость	95% интервал	Без кеша
SR – SN	997	0,986	0,939	[0,93; 0,95]	0,955
MR – MN	979	0,990	0,978	[0,97; 0,99]	0,986
MN – SN	983	3,078	2,672	[2,63; 2,71]	2,777
MR – SR	986	3,090	2,780	[2,74; 2,82]	2,867

Отношения делят суммы трехповторных средних по задачам на полных ресурсных парах. Показанный интервал относится к отношению стоимостных оценок по тарифам API. Вариант «Без кеша» устраняет скидку на кешированный ввод; абонентская плата не оценивается.

Различия между колонками токенов и стоимости связаны с составом входа и выхода и со скидкой на кешированный ввод. Поэтому меньшее число токенов и меньшая денежная оценка не являются взаимозаменяемыми показателями.

5.4 Сопоставление с более ранней серией

Более раннее факторное исследование GPT-5.4-mini использовало те же 200 идентификаторов задач, позднее сохраненных в расширенном распределении, но создавало по одному независимому запросу на условие с прежней моделью. Из 1 000 назначений завершились 996; контрасты включают 191–193 допустимые пары задач. Все четыре оцененных эффекта имеют то же направление, что и в исследовании Luna (табл. 8): эффекты ролевых меток отрицательны при одном вызове и положительны при трех, тогда как эффекты структуры вызовов отрицательны при обоих вариантах меток. После поправки Холма ни один из четырех тестов не отвергает нулевую гипотезу. Подтверждение относится к совпадению направления для двух псевдонимов моделей на тех же 200 задачах, а не к независимой выборке задач или повторной проверке с тем же объемом данных, что в основной серии. В более ранней серии отношения стоимостной оценки трех вызовов равны 2,09 относительно SN и 2,50 относительно SR.

Таблица 8. Более ранняя серия GPT-5.4-mini на 200 повторно использованных идентификаторах задач. Разности и описательные интервалы указаны в процентных пунктах; *n* — число полных пар задач. Четыре точных теста Макнемара образуют семейство с поправкой Холма.

Контраст	<i>n</i>	Δ	95% интервал	<i>p</i> Холма
SR–SN	191	−1,05	[−4,71; +2,62]	1,000
MR–MN	193	+1,04	[−3,11; +5,70]	1,000
MN–SN	191	−4,71	[−9,42; 0,00]	0,313
MR–SR	193	−2,07	[−6,22; +2,07]	1,000

Точный тест Макнемара в этой серии сравнивает парные бинарные исходы: он использует задачи, на которых успешным оказалось только одно из двух условий, и проверяет симметрию таких расхождений. Это отличается от основного анализа средних трех повторов, для которого применен t -тест.

Другие сохраненные пилотные эксперименты варьируют размещение инструкций или меняют процедуру рассуждения. Мы не объединяем их с основным исследованием.

6 СРАВНЕНИЕ С SELF-COLLABORATION

Мы сравниваем Self-Collaboration (SCC) с одношаговыми условиями, используя ту же модель Luna, средний уровень рассуждения и штатный оценщик. Это сравнение относится к полным процессам. Адаптированный авторский контроллер [4] начинает с плана анализа в JSON — текстовом формате структурированных данных, вызывает разработчика для получения Python-кода, а затем просит тестировщика сгенерировать до пяти примеров вызова в функции `check(candidate)`. Он выполняет программу и сгенерированную проверку; внутренним сигналом остановки служит выполнение без исключений, при этом напечатанные значения автоматически не сравниваются с ожидаемыми ответами. Заданный предел допускает одну первоначальную реализацию и не более одного исправления. Модель получает полный предоставленный контекст задачи и историю сообщений ролей, записанную в текстовом формате. Эталонные решения и тесты бенчмарка остаются внешними.

Первоначальный план содержит 3 000 блоков «задача–повтор», в каждом из которых назначаются новые процессы SR, SN и SCC. Поправка, принятая после начала генерации, но до проверки качества SCC, фиксирует первые 2 000 рандомизированных блоков (seed порядка 20260911). Это дает 6 000 назначений методов на 956 задачах: у 212 задач выбран один повтор, у 444 — два, у 300 — три. Из этих задач 941 проходит эталонные проверки. Невыбранные 3 000 назначений методов не входят в знаменатель после поправки.

Штатное оценивание прошли все 5 595 завершенных программ: 1 997 SR, 1 998 SN и 1 600 SCC. В сравнении сохранены 397 инфраструктурных сбоев и восемь ранее приостановленных попыток. В частности, у SCC имеется 396 сбоев Docker — среды запуска изолированных контейнеров — и четыре приостановленных процесса; у SR — три приостановленные попытки; у SN — одна приостановленная попытка и один сбой потока. Ни одна отправленная попытка не генерировалась повторно. После эталонных проверок качество наблюдается для 1 964 повторов SR, 1 965 повторов SN и 1 574 повторов SCC; число успехов равно соответственно 1 067, 1 064 и 809. Это количества наблюдаемых повторов, а не независимых задач и не показатели выбора лучшего из трех.

Для каждого сравниваемого метода исходы SCC минус исходы сравниваемого метода сначала усредняются по совпадающим выбранным повторам, в которых наблюдаются оба исхода, а затем с равными весами по допустимым задачам, имеющим хотя бы одну такую пару. Поэтому задача с тремя выбранными повторами имеет тот же вес, что и задача с одним повтором. Двусторонние t -тесты на уровне задач для SCC–SR и SCC–SN образуют отдельное семейство из двух тестов с поправкой Холма. Интервалы используют 10 000 выборок целых задач, генератор PCG64, seed 20260911 и линейные процентильные границы.

Таблица 9 показывает парные разности на задачах с наблюдаемыми исходами. Отрицательный знак означает меньшую долю успеха SCC относительно указанного одношагового процесса.

Таблица 9. Качество SCC на наблюдаемых сопоставленных повторях. Разности и интервалы указаны в процентных пунктах; n — число участвующих задач.

Контраст	n	Δ	95% ДИ	p Холма
SCC–SR	883	–2,68	[–4,72; –0,60]	0,0217
SCC–SN	882	–2,48	[–4,50; –0,43]	0,0217

Оба условных контраста отрицательны (табл. 9). Интервалы чувствительности по группам исходных задач сходны: [–4,72; –0,58] пункта относительно SR и [–4,51; –0,43] относительно SN для объединенного разбиения. Для оценки всей назначенной совокупности отдельно учтены пропуски. Их крайние допустимые значения задают границы разности: каждому неизвестному исходу приписывается сначала наиболее благоприятное, затем наименее благоприятное для SCC значение. Это расчет границ, а не восстановление фактических ответов. Экстремальные границы с фиксированным знаменателем по всем 941 допустимым выбранным задачам равны [–12,98; +6,61] и [–13,14; +6,43] пункта. Первая граница означает, что относительно SR данные допускают как проигрыш SCC до 12,98 процентного пункта, так и выигрыш до 6,61 пункта. Вторая аналогично относится к SN. Таким образом, вся совокупность охвачена интервальной оценкой, а наблюдаемые пары — более точной условной оценкой. Границы не являются доверительными интервалами: они отражают неизвестные исходы, а не случайный разброс наблюдаемой выборки. Однозначное ранжирование всей совокупности потребовало бы доступных итоговых результатов незавершенных процессов либо дополнительных предположений о пропусках.

Для сравнения ресурсов необходимы оба полностью завершённых процесса с корректными счетчиками. Отношение парных средних по задачам для API-эквивалентной стоимостной оценки равно 3,138 относительно SR (896 задач; 95%-й интервал [3,067; 3,208]) и 2,939 относительно SN (895 задач; [2,871; 3,010]). Эти интервалы отношений средних используют 10 000 выборок целых задач с seed 20260914. Они относятся к наблюдаемым полным процессам и не объединяют сохранение качества с денежным преимуществом. Доступность и сопоставленные эффекты до и после поправки сохранены как описательные проверки; они не идентифицируют причинный временной эффект.

Отдельная диагностика оценивает все 399 сохранённых программ разработчика из незавершённых процессов SCC, не перезапуская контроллер и не добавляя сгенерированные им проверки. Среди 392 программ, допустимых по контролям, она находит 196 успешных; семь программ не проходят контроль допустимости. Эти результаты промежуточных программ не заменяют неизвестные исходы полного процесса в сравнении.

7 ОБСУЖДЕНИЕ

Завершённое факторное исследование оценивает условную разность между проверенными процессами. В протоколе полного контекста без сжатия распределение планирования, реализации и критики между тремя отдельными последовательными вызовами увеличивает повторную передачу и измеренный вывод и даёт меньшую успешность штатных тестов на наблюдаемых парах задач, чем размещение тех же операций в одном ответе. Результат относится к исследованной структуре вызовов, модели, совокупности задач, пределу времени и контракту оценивания. Он не показывает, что отдельные агенты в общем случае неэффективны или что дополнительное взаимодействие не может помочь, когда у агентов есть инструменты, поиск, состояние репозитория или выровненный вычислительный бюджет.

Вмешательство ролевых меток меньше и неопределеннее. Оценка роли при трех вызовах описательно положительна, но не достигает скорректированного по Холму порога значимости в основном анализе. Оценки неточны и имеют разные знаки при разных структурах вызовов, поэтому не устанавливают преимущества от именования этапов промпта в этом режиме. Неотвержение также не устанавливает эквивалентность качества. Внутренние метки все же могут организовать промпт или передать полезную декомпозицию; эксперимент измеряет, изменили ли они исход бенчмарка при одинаковых формулировках операций.

Использование ресурсов следует интерпретировать вместе с качеством. Одношаговые ролевые метки немного снижают измеренную стоимостную оценку в этом исследовании, тогда как три вызова стоят примерно в 2,7–2,8 раза дороже. Поскольку дизайн не задает строгий выровненный бюджет вывода, учет включает обусловленную длину рассуждения, повторный контекст, промежуточные ответы и состояние кеша. Заявление о совместном превосходстве по качеству и стоимости не делается. Анализ без кеша изменяет величину, но сохраняет большой контраст топологии.

Внешний оценщик также не позволяет процессу получить зачет за убедительное само-описание исправления, которое не изменило исполнимое поведение. В штатные тесты поступает только итоговый кандидат. У прямого условия наименьшие наблюдаемые средние затраты ресурсов. Его оцененные различия качества с одношаговыми поэтапными условиями малы, а интервалы включают ноль; эти разведочные тесты не предназначены для установления эквивалентности. На наблюдаемых парах задач прямое условие успешнее обоих трехшаговых условий, хотя эти сравнения не были заранее зарегистрированы как подтверждающее семейство.

В рассматриваемом режиме прямой и одношаговый поэтапный процессы являются менее дорогими базовыми линиями. Дополнительные вызовы следует оценивать, когда они добавляют параллельный поиск, сохраненное состояние рабочей среды, инструменты, извлечение данных или исполнимую обратную связь. При оценивании следует сообщать и парное по задачам качество по результатам тестов, и измеренное число токенов с указанием модели и правил учета.

Три повтора улучшают оценку для выбранных задач, но не превращают 15 000 назначений в независимые единицы-задачи. Бутстреп сохраняет повторы вместе, а каждая задача вносит среднюю успешность на генерацию с одинаковым весом. Это не позволяет считать повторные запросы новыми единицами-задачами или выбирать лучший результат из трех. Процесс с отдельными вызовами все еще может быть полезен при наличии инструментов, поиска, состояния репозитория, параллельных подзадач, восстановления или выровненного вычислительного бюджета; эти возможности находятся за пределами эксперимента с фиксированным контекстом.

Вспомогательные исследования по одному изменяют модель, размещение инструкций или процедуру рассуждения. Их результаты отделены от тестов качества Luna и не устраняют возможность загрязнения обучающими данными бенчмарка.

8 ОГРАНИЧЕНИЯ И ВОСПРОИЗВОДИМОСТЬ

Работа использует два псевдонима моделей, каждый с одним уровнем рассуждения и одной версией клиента, поэтому различия между целыми семействами моделей по этим данным не оцениваются. Под псевдонимом понимается доступное пользователю имя модели, которому поставщик может сопоставлять обновляемую внутреннюю версию. Меньшая серия повторно использует те же 200 идентификаторов задач и имеет только один запрос на

условие; совпадение направления является подтверждением, а не независимой репликацией. Поставщик не раскрыл неизменяемый идентификатор весов, значение температуры или seed выборки. Расширенная выборка повторно использует эти 200 идентификаторов, поэтому не является 1 000 полностью новыми отложенными задачами. Задачи опубликованы открыто, поэтому нельзя исключить их присутствие в обучающих данных модели; это называют загрязнением оценки. Похожие задачи также могут давать связанные результаты и уменьшать фактическое число независимых наблюдений.

Качество остается неизвестным для недоступных генераций и инфраструктурных сбоев. Наблюдаемые сбои тестов и превышения времени бенчмарка считаются неуспешными программами на допустимых задачах. После транспортных прерываний продолжение разрешалось только для еще не начатых назначений; начатые вызовы не повторялись. Оценки по доступным случаям относятся к наблюдаемому допустимому подмножеству. Экстремальные границы по назначениям показывают, что допускают отсутствующие бинарные исходы, но не устраняют неслучайные пропуски. Поэтому исследование избегает заявлений об эквивалентности, не меньшей эффективности или экономически оптимальном дизайне.

Штатные тесты дают исполнимое свидетельство, но наследуют связь между спецификациями и тестами бенчмарка. Эталонный и некорректный контроли выявляют непригодные окружения и нечувствительные негативные проверки; они не доказывают, что каждое утверждение отражает задачу на естественном языке. Протокол фиксированного контекста исключает поиск, инструменты, взаимодействие с репозиторием и адаптивную обратную связь от тестов. Следовательно, результаты не устанавливают качество исправления репозитория или сквозной разработки программного обеспечения.

Вмешательство уже, чем более широкий прототип, который его мотивировал: выбор контекста, выборочные перезапуски, вспомогательная проверка и сжатие отключены. Это изолирует ядро поэтапной генерации, но не заменяет полную оценку развертывания. Сравнение SCC имеет дополнительное ограничение, поскольку его сгенерированные проверки и сигнал остановки при отсутствии исключений задают другой контроллер обратной связи. Более высокая стоимость и меньшая успешность SCC на наблюдаемых парах задач относятся ко всему процессу и не изолируют вклад ролевых меток внутри SCC. Значительные инфраструктурные пропуски SCC ограничивают выводы на уровне совокупности.

Идентификаторы задач, промпты, ответы, состояния, флаги извлечения, счетчики, контроли, отчеты штатной проверки и seed анализа сохранены в материалах проекта. Эти записи позволяют восстановить знаменатели назначений и парные подмножества. Новая ревизия поставщика, версия зависимости или интерпретация связи задачи и теста может изменить доли. При воспроизведении следует сохранить указанные псевдоним модели, версию клиента, байты контекста, пределы времени, контрольный фильтр и правила пропусков и сообщать любое отклонение.

9 ЗАКЛЮЧЕНИЕ

Работа разделяет два проектных решения в генерации кода: название этапов и их распределение по вызовам. Цель достигнута в пределах зафиксированных моделей, задач и правил проверки: для каждого решения получены отдельные оценки качества и ресурсов, а для отсутствующих исходов рассчитаны границы возможного результата.

Ответ на RQ1. В основной серии ролевые метки изменили долю успеха на $-0,75$ процентного пункта при одном вызове и на $+1,35$ пункта при трех. После поправки Холма ни один из двух тестов не устанавливает преимущества по качеству. При этом описательные

отношения стоимостных оценок равны 0,939 и 0,978 соответственно. Метки задают структуру инструкции, но проверка не подтверждает, что одно лишь именование этапов устойчиво улучшает качество; ресурсные различия следует оценивать вместе с этой неопределенностью.

Ответ на RQ2. Три вызова вместо одного снизили парную долю успеха на 4,17 пункта для нейтральных инструкций и на 2,13 пункта для рольевых, увеличив стоимостную оценку в 2,67 и 2,78 раза. Для обоих сравнений направление качества сохраняется даже при крайних допустимых исходах пропусков. Оно также сохраняется в анализе 800 новых задач и совпадает с направлением более ранней серии на 200 задачах. Вывод относится к совместно изменению повторной передачи контекста, промежуточных ответов и объема генерации в данном протоколе.

Ответ на RQ3. Различия качества прямого решателя и одношаговых поэтапных процессов малы и неопределенны, а его средний наблюдаемый расход ресурсов ниже. На наблюдаемых парах он успешнее обоих трехшаговых процессов; эти сравнения имеют разведочный характер. В отдельной серии SCC уступает одношаговым процессам по качеству на наблюдаемых парах и имеет примерно вдвое большую стоимостную оценку. Для всей назначенной совокупности рассчитанные границы качества SCC включают как преимущество, так и проигрыш, поэтому полный вывод состоит из условной оценки и явно указанного диапазона неопределенности.

Практическое следствие — использовать прямой и одношаговый поэтапный процессы как обязательные экономичные варианты сравнения в задачах с полным фиксированным контекстом. Добавление вызовов целесообразно обосновывать измеряемой пользой: например, новыми сведениями, поиском, инструментами или обратной связью. Дальнейшая проверка должна отдельно исследовать эти механизмы, более широкий набор моделей и выполнение задач в репозиториях. Полученные результаты не заменяют таких экспериментов.

ДОСТУПНОСТЬ ДАННЫХ

К рукописи приложен анонимизированный пакет воспроизводимости. Он содержит зафиксированные протоколы и поправки, выборки задач, исходы на уровне назначений, счетчики состояния и использования, основные программы-кандидаты, все основные отчеты штатной проверки, свидетельства контролей, спецификации зафиксированного окружения, исходный код и данные оценщика бенчмарка под их первоначальной лицензией, разбиения по зависимости исходных задач, скрипты анализа, точные входы таблиц и манифест хешей файлов. Пакет позволяет повторно выполнить численные анализы основного, мини- и SCC-исследований и компонентов исходных задач; проверить главные табличные значения; а также подтвердить, что основные программы и состояния штатной проверки точно совпадают с неизменяемыми записями оценщика без учетных данных поставщика. Полные тела диалоговых ответов, исходный текст кандидатов SCC и рабочие транспортные трассы не включены, поскольку не нужны для этих проверок и могут содержать случайные сгенерированные идентификаторы или локальные метаданные машины. Исключены история репозитория, способная раскрыть личности авторов, локальные пути, идентификаторы учетных записей и учетные данные. Полный сохраненный архив будет опубликован после периода двойного анонимного рецензирования с учетом условий бенчмарка и поставщика.

СПИСОК ЛИТЕРАТУРЫ

- [1] Miltiadis Allamanis. 2019. The Adverse Effects of Code Duplication in Machine Learning Models of Code. In *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*. ACM, 143–153. doi:10.1145/3359591.3359735
- [2] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. 2023. CodeT: Code Generation with Generated Tests. In *The Eleventh International Conference on Learning Representations*. <https://openreview.net/forum?id=ktw68Cmu9c>
- [3] Junhao Chen, Xiang Li, Yibin Xu, Yuehan Cui, Fangsheng Weng, Hao Zhao, Fei Ma, and Qi Tian. 2026. PairCoder: Pair Programming-Inspired Two-Agent Collaboration for Code Generation. In *Findings of the Association for Computational Linguistics: ACL 2026*. Association for Computational Linguistics, 3043–3058. doi:10.18653/v1/2026.findings-acl.149
- [4] Yihong Dong. 2024. *Self-Collaboration Code Generation: Accompanying Python Software*. Documented author implementation associated with Dong et al.; analyst–coder–tester session with generated-check execution. Historical 2024 analyst–coder–tester implementation, associated with arXiv:2304.07590; workflow and transport adaptations are specified in the text.. <https://github.com/YihongDong/Self-collaboration-Code-Generation>
- [5] Yihong Dong, Xue Jiang, Zhi Jin, and Ge Li. 2024. Self-Collaboration Code Generation via ChatGPT. *ACM Transactions on Software Engineering and Methodology* 33, 7 (2024), 1–38. Preprint: arXiv:2304.07590. doi:10.1145/3672459
- [6] Junda He, Christoph Treude, and David Lo. 2025. LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision, and the Road Ahead. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025), 1–30. doi:10.1145/3712003
- [7] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiwu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=VtmBAGCN7o>
- [8] Yunsong Hong. 2026. Learning How Much to Collaborate: Difficulty-Aware Topology Selection for Multi-Agent Code Generation. arXiv:2609.13890 <https://arxiv.org/abs/2609.13890>
- [9] Dong Huang, Jie M. Zhang, Michael Luck, Qingwen Bu, Yuhao Qing, and Heming Cui. 2023. AgentCoder: Multi-Agent-based Code Generation with Iterative Testing and Optimisation. arXiv:2312.13010 doi:10.48550/arXiv.2312.13010
- [10] Md. Ashraful Islam, Mohammed Eunus Ali, and Md Rizwan Parvez. 2024. MapCoder: Multi-Agent Code Generation for Competitive Problem Solving. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 4912–4944. doi:10.18653/v1/2024.acl-long.269
- [11] Sayash Kapoor, Benedikt Stroebel, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. 2025. AI Agents That Matter. *Transactions on Machine Learning Research* (2025). <https://openreview.net/forum?id=Zy4uFzMviZ>
- [12] Pedro Henrique Luz de Araujo, Paul Röttger, Dirk Hovy, and Benjamin Roth. 2025. Principled Personas: Defining and Measuring the Intended Effects of Persona Prompting on Task Performance. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 26857–26886. doi:10.18653/v1/2025.emnlp-main.1364
- [13] James G. MacKinnon, Morten Ørregaard Nielsen, and Matthew D. Webb. 2023. Cluster-Robust Inference: A Guide to Empirical Practice. *Journal of Econometrics* 232, 2 (2023), 272–299. doi:10.1016/j.jeconom.2022.04.001
- [14] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems*, Vol. 36. doi:10.52202/075280-2019
- [15] OpenAI. 2026. *API Pricing*. Standard text-token rates. <https://developers.openai.com/api/docs/pricing>
- [16] OpenAI. 2026. *Codex Non-Interactive Mode and JSONL Event Stream*. <https://developers.openai.com/codex/noninteractive>
- [17] OpenAI. 2026. *GPT-5.4 Mini Model Documentation*. <https://developers.openai.com/api/docs/models/gpt-5.4-mini>
- [18] OpenAI. 2026. *GPT-5.6 Luna Model Documentation and Pricing*. <https://developers.openai.com/api/docs/models/gpt-5.6-luna>
- [19] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ChatDev: Communicative Agents for Software Development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 15174–15186. doi:10.18653/v1/2024.acl-long.810
- [20] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*, Vol. 36. doi:10.52202/075280-0377

- [21] Haoran Sun and Shaoning Zeng. 2025. Introspection of Thought Helps AI Agents. arXiv:2507.08664 <https://arxiv.org/abs/2507.08664>
- [22] Dat Tran and Douwe Kiela. 2026. Single-Agent LLMs Outperform Multi-Agent Systems on Multi-Hop Reasoning Under Equal Thinking Token Budgets. arXiv:2604.02460 <https://arxiv.org/abs/2604.02460>
- [23] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2025. Demystifying LLM-Based Software Engineering Agents. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 801–824. doi:10.1145/3715754
- [24] Jiawei Xu, Arief Koesdwiady, Sisong Bei, Yan Han, Baixiang Huang, Dakuo Wang, Yutong Chen, Zheshen Wang, Peihao Wang, Pan Li, and Ying Ding. 2026. Rethinking the Value of Multi-Agent Workflow: A Strong Single Agent Baseline. arXiv:2601.12307 <https://arxiv.org/abs/2601.12307>
- [25] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. Published as a NeurIPS 2024 paper; arXiv version retained for stable metadata. arXiv:2405.15793 <https://arxiv.org/abs/2405.15793>
- [26] Huan Zhang, Wei Cheng, Yuhan Wu, and Wei Hu. 2024. A Pair Programming Framework for Code Generation via Multi-Plan Exploration and Feedback-Driven Refinement. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. ACM, 1319–1331. doi:10.1145/3691620.3695506
- [27] Mingqian Zheng, Jiaxin Pei, Lajanugen Logeswaran, Moontae Lee, and David Jurgens. 2024. When “A Helpful Assistant” Is Not Really Helpful: Personas in System Prompts Do Not Improve Performances of Large Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. Association for Computational Linguistics, 15126–15154. doi:10.18653/v1/2024.findings-emnlp.888
- [28] Li Zhong, Zilong Wang, and Jingbo Shang. 2024. Debug like a Human: A Large Language Model Debugger via Verifying Runtime Execution Step-by-Step. In *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, 851–870. doi:10.18653/v1/2024.findings-acl.49
- [29] Yueheng Zhu, Chao Liu, Xuan He, Xiaoxue Ren, Zhongxin Liu, Ruwei Pan, and Hongyu Zhang. 2026. AdaCoder: An Adaptive Planning and Multi-Agent Framework for Function-Level Code Generation. *IEEE Transactions on Software Engineering* 52, 2 (2026), 631–650. doi:10.1109/TSE.2025.3642621
- [30] Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widyasari, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, Simon Brunner, Chen Gong, Thong Hoang, Arnel Randy Zebaze, Xiaoheng Hong, Wen-Ding Li, Jean Kaddour, Ming Xu, Zhihan Zhang, Prateek Yadav, Naman Jain, Alex Gu, Zhoujun Cheng, Jiawei Liu, Qian Liu, Zijian Wang, David Lo, Binyuan Hui, Niklas Muennighoff, Daniel Fried, Xiaoning Du, Harm de Vries, and Leandro von Werra. 2025. BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions. In *International Conference on Learning Representations*. Published as a conference paper at ICLR 2025. <https://arxiv.org/abs/2406.15877>